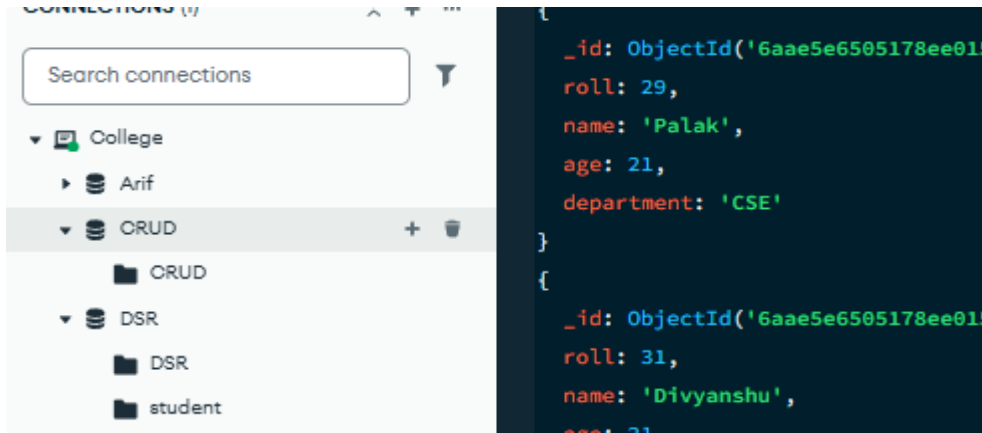


1. CREATE DATABASE

MongoDB Query:

use CRUD

Explanation: Creates or switches to the CRUD database.

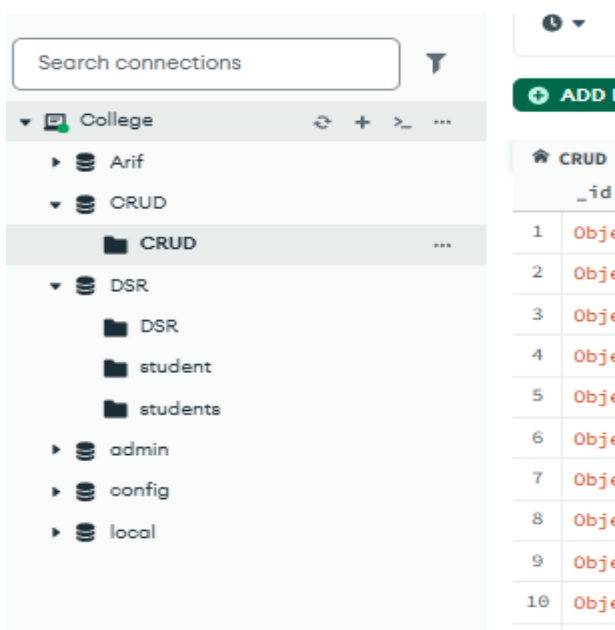


2. CREATE COLLECTION

MongoDB Query:

db.create Collection("CRUD")

Explanation: Creates a collection named CRUD.



3. DISPLAY ALL STUDENTS

MongoDB Query:

db. CRUD. Find ().

```
}  
> db.CRUD.find()  
< {  
  _id: ObjectId('6a9e544e85178ee815c9caf2'),  
  rollno: 1,  
  name: 'Sam',  
  age: 28,  
  department: 'ECA'  
}  
{  
  _id: ObjectId('6a9e5e6585178ee815c9caf3'),  
  rollno: 2,  
  name: 'Ram',  
  age: 28,  
  department: 'ECA'  
}  
{  
  _id: ObjectId('6a9e5e6585178ee815c9caf4'),  
  roll: 3,  
  name: 'Adi',  
  age: 28,  
  department: 'ECA'  
}
```

Explanation: Retrieves documents from the collection

4. EQUAL TO — \$eq

MongoDB Query:

db.students.find({marks: {\$eq: 78}})

```
> db.students.find({marks: {$eq: 78}})
< {
  _id: ObjectId('6ab21b30555468477e03bd32'),
  name: 'Rohit',
  age: 21,
  marks: 78,
  department: 'CSE',
  skills: [
    'Python',
    'AI'
  ]
}
```

Explanation: Finds students whose marks are exactly 78.

5. GREATER THAN — \$gt

MongoDB Query:

`db.students.find({marks: {$gt: 85}})`

```
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Anamika',
  age: 19,
  marks: 88,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Vaishnavi',
  age: 19,
  marks: 92,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Pratik',
  age: 19,
  marks: 90,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2227'),
  roll: 12,
  name: 'Shreya',
  age: 19,
  marks: 95,
  city: 'Lucknow'
}
```

Explanation: Finds students whose marks are greater than 85.

6. LESS THAN — \$lt

MongoDB Query:

db.students.find({marks: {\$lt: 80}})

```
> db.students.find({marks:{$lt:80}})
< {
  _id: ObjectId('6ab21b30555468477e83bd2f'),
  name: 'Rahul',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e83bd31'),
  name: 'Neha',
  age: 23,
  marks: 65,
  department: 'ME',
  skills: [
    'AutoCAD',
    'Python'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e83bd32'),
  name: 'Rohit',
  age: 21,
  marks: 78,
  department: 'CSE',
  skills: [
    'Python',
    'AI'
  ]
}
```

Explanation: Finds students whose marks are less than 80.

7. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

db.students.find({marks: {\$gte: 85}})

```
> db.students.find({marks: {$gte: 85}})
< {
  _id: ObjectId('6ab21b30555468477e03bd2e'),
  name: 'Amit',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e03bd30'),
  name: 'Priya',
  age: 20,
  marks: 91,
  department: 'CSE',
  skills: [
    'Java',
    'MongoDB',
    'AI'
  ]
}
```

Explanation: Finds students whose marks are 90 or above.

8. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

db.students.find({marks: {\$lte: 85}})

```
> db.students.find({marks: {$lte: 85}})
< {
  _id: ObjectId('6ab21b30555468477e03bd2e'),
  name: 'Amit',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e03bd2f'),
  name: 'Rahul',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
```

Explanation: Finds students whose marks are 85 or below.

9. UPDATE ONE OPERATION

MongoDB Query:

```
db.CRUD.updateOne(  
  {roll:31},  
  {$set:{department:"ECA"}}  
)
```

```
> db.CRUD.updateOne({roll:31},{ $set:{department:"ECA"}})  
< {  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}
```

```
{  
  _id: ObjectId('6a9e5e6585178ee015c9cb18'),  
  roll: 31,  
  name: 'Divyanshu',  
  age: 21,  
  department: 'ECA'  
}
```

Explanation: Updates the department to ECA of the student with roll number 31.

10. DELETE ONE OPERATION

MongoDB Query:

```
db.CRUD.deleteOne({roll:15})
```

```
> db.CRUD.deleteOne({roll:15})
{
  acknowledged: true,
  deletedCount: 1
}
> db.CRUD.find()
```

Explanation: Deletes one document matching roll number 10.

12. DELETE MANY OPERATION

MongoDB Query:

```
db.CRUD.deleteMany({
  department:"ME"
})
```

```
> db.CRUD.deleteMany({department:"ME"})
{
  acknowledged: true,
  deletedCount: 16
}
> db.CRUD.find()
```

Explanation: Deletes all students having marks below 60.

11. AND OPERATION — \$and

MongoDB Query:

```
db.students.find({
  $and:[
    {marks:{$gt:70}},
    {age:23}
  ]
})
```

```
> db.students.find({$and:[{marks:{$gt:70}},{age:{$lt:23}}]})
< {
  _id: ObjectId('6ab21b30555468477e03bd2e'),
  name: 'Amit',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e03bd2f'),
  name: 'Rahul',
  age: 22,
  marks: 72,
  department: 'ECE',
  skills: [
    'Python',
    'SQL'
  ]
}
```

Explanation: Finds students satisfying both conditions.

12. OR OPERATION — \$or

MongoDB Query:

```
db.students.find({
  $or:[
    {department:"CSE"},
    { department:"ECA "}
  ]
})
```

```
> db.students.find({$or:[{department:"CSE"},{department:"ECA"}]})
{
  _id: ObjectId('6ab21b30555468477e03bd2e'),
  name: 'Amit',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e03bd30'),
  name: 'Priya',
  age: 20,
  marks: 91,
  department: 'CSE',
  skills: [
    'Java',
    'MongoDB',
    'AI'
  ]
}
```

Explanation: Finds students from CSE or ECA.

MONGODB OPERATORS USED:

Operator	Meaning	Example
insertOne insertMany	Create one and many	insertOne({name:"Arif"}) insertMany([{name:"Adeel"},{name:"Arif"}])
Find()	Read	Find()
updateOne updateMany	Update one and many	updateOne({},{}) updateMany({},{})
deleteOne deleteMany	Delete one and many	Delete One({}) Delete Many({})
\$eq	Equal to	marks: {\$eq:88}
\$gt	Greater than	marks: {\$gt:85}
\$lt	Less than	marks: {\$lt:60}
\$gte	Greater than or equal to	marks: {\$gte:90}
\$lte	Less than or equal to	marks: {\$lte:70}
\$and	All conditions true	\$and:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.